

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»**

**Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії**

Звіт до комп'ютерного практикуму №2
з навчальної дисципліни «Розробка мобільних застосунків під Android»
Тема: ДОСЛІДЖЕННЯ РОБОТИ З КОМПОНЕНТОМ FRAGMENT
Варіант 8

Виконала:
Студентка групи ІІІ-34
Литовченко Ангеліна

Київ – 2026

1. Мета роботи

Дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність (Activity) та набуті практичні навички з використання фрагментів для побудови інтерфейсу користувача Android-застосунку.

2. Завдання (варіант 8)

Написати програму під платформу Android, яка реалізує наступний інтерфейс:

Вікно містить два текстових поля для введення чисел, групу радіо-кнопок для вибору арифметичної операції (+, −, ×, ÷) та кнопку «ОК». При натисканні «ОК» результат обчислення відображається у другому фрагменті. Якщо не всі дані введені або не вибрано операцію - відображається Toast-повідомлення. При спробі ділення на нуль виводиться відповідне попередження.

Програма реалізована з використанням двох фрагментів у рамках однієї Activity:

- FirstFragment - форма введення (два поля EditText, RadioGroup з чотирма операціями, кнопка ОК);
- SecondFragment - відображення результату (TextView) та кнопка Cancel для повернення і очищення форми.

Передача даних між фрагментами реалізована через SharedViewModel.

3. Опис реалізації

3.1. Структура проєкту

Файл	Призначення
MainActivity.kt	Єдина Activity; завантажує FirstFragment при старті
FirstFragment.kt	Фрагмент введення даних і запуску обчислення
SecondFragment.kt	Фрагмент відображення результату з кнопкою Cancel
SharedViewModel.kt	ViewModel для передачі даних між фрагментами
activity_main.xml	FrameLayout - контейнер для фрагментів

fragment_first.xml	Макет першого фрагменту (EditText, RadioGroup, Button)
fragment_second.xml	Макет другого фрагменту (TextView, Button Cancel)

3.2. MainActivity

Activity містить єдиний FrameLayout з ідентифікатором fragment_container. При першому запуску завантажується FirstFragment:

```
if (savedInstanceState == null) {
    supportFragmentManager.beginTransaction()
        .replace(R.id.fragment_container, FirstFragment())
        .commit()
}
```

3.3. SharedViewModel

Містить два поля LiveData: resultText для передачі результату обчислення та shouldClear - прапор-сигнал для очищення форми введення. Завдяки activityViewModels() обидва фрагменти отримують один і той самий екземпляр ViewModel:

```
class SharedViewModel : ViewModel() {
    val resultText = MutableLiveData<String>()
    val shouldClear = MutableLiveData<Boolean>(false)
}
```

3.4. FirstFragment

При натисканні кнопки ОК виконується перевірка заповненості полів та наявності вибраної операції (Toast при помилці), перевірка ділення на нуль, обчислення результату через when-вираз, запис у ViewModel та перехід до SecondFragment:

```
val result = when (checked) {
    R.id.radioPlus -> val1 + val2
    R.id.radioMinus -> val1 - val2
    R.id.radioMult -> val1 * val2
    R.id.radioDiv -> val1 / val2
    else -> 0.0
}
viewModel.resultText.value = "$val1 $operationSign $val2 = $result"
parentFragmentManager.beginTransaction()
    .replace(R.id.fragment_container, SecondFragment())
    .addToBackStack(null)
```

```
.commit()
```

Фрагмент також підписаний на shouldClear через observe. Коли SecondFragment виставляє цей прапор у true, FirstFragment очищує поля і скидає прапор:

```
viewModel.shouldClear.observe(viewLifecycleOwner) { clear ->
    if (clear == true) {
        num1.text.clear()
        num2.text.clear()
        group.clearCheck()
        viewModel.shouldClear.value = false
    }
}
```

3.5. SecondFragment

Підписується на resultText і відображає результат у TextView. При натисканні Cancel виставляє shouldClear = true та повертається до FirstFragment через popBackStack(). Вся комунікація між фрагментами відбувається виключно через ViewModel - без прямих посилань між фрагментами:

```
btnCancel.setOnClickListener {
    viewModel.resultText.value = ""
    viewModel.shouldClear.value = true
    parentFragmentManager.popBackStack()
}
```

3.6. Макети

fragment_first.xml - LinearLayout з двома EditText (inputType numberDecimal|numberSigned для підтримки від'ємних чисел), RadioGroup з чотирма RadioButton (+, -, ×, ÷) та кнопкою OK.

fragment_second.xml - LinearLayout з TextView (textStyle bold) для відображення результату та кнопкою Cancel, вирівняними по центру.

activity_main.xml - FrameLayout як єдиний контейнер для підміни фрагментів.

4. Відповіді на контрольні питання

1. Призначення та можливості компонента Fragment

Fragment - це модульна частина інтерфейсу користувача або поведінки Activity, яка має власний макет, власний стек викликів і власний життєвий цикл. Фрагменти дозволяють будувати адаптивний UI: наприклад, на смартфоні показувати один фрагмент, на планшетах - два

одночасно. Один і той самий фрагмент може бути повторно використаний у різних Activity. Підтримують ViewModel, LiveData та Navigation Component.

2. Життєвий цикл Fragment

Фрагмент проходить такі стани: `onAttach()` → `onCreate()` → `onCreateView()` → `onViewCreated()` → `onStart()` → `onResume()` → `onPause()` → `onStop()` → `onDestroyView()` → `onDestroy()` → `onDetach()`. При використанні Back Stack після `onStop()` фрагмент призупиняється до `onDestroyView()`, але не знищується повністю, що дозволяє відновити стан при поверненні.

3. Способи створення Fragment

Статично - оголошення тегу `<fragment>` або `<FragmentContainerView>` безпосередньо в XML-макеті Activity.

Динамічно - через `FragmentManager` у коді: `supportFragmentManager.beginTransaction().add()` або `.replace()` з подальшим `.commit()`. Також підтримується `Navigation Component` з графом навігації.

4. Способи управління Fragment

Управління здійснюється через `FragmentManager` та `FragmentManager`.

Основні операції: `add()` - додати фрагмент у контейнер,
`replace()` - замінити поточний фрагмент,
`remove()` - видалити,
`hide()/show()` - приховати або показати без знищення.

Метод `addToBackStack()` дозволяє повертатись до попереднього фрагменту натисканням системної кнопки «Назад».

5. Способи взаємодії між фрагментами

Через спільну `ViewModel` - фрагменти отримують один екземпляр `ViewModel` через `activityViewModels()` і обмінюються даними через `LiveData`.

Через `Fragment Result API` - `setFragmentResult()` / `setFragmentResultListener()` для передачі разових результатів.

Через інтерфейс-слухач - фрагмент оголошує інтерфейс, Activity реалізує і передає посилання.

Через Bundle-аргументи - `Fragment.newInstance()` з параметрами при створенні.

6. Поняття системи, малої системи та мобільної платформи

Система - сукупність взаємопов'язаних елементів, що функціонують як єдине ціле для досягнення певної мети. Мала система - система з обмеженими ресурсами: обчислювальними, пам'яттю та живленням (наприклад, мобільний пристрій). Мобільна платформа - програмно-апаратне середовище для мобільних пристроїв, що включає операційну систему, набір API та інструменти розробки (Android від Google, iOS від Apple).

7. Типи мобільних застосунків

Нативні застосунки - розробляються для конкретної платформи (Kotlin/Java для Android, Swift для iOS), мають доступ до всіх API пристрою та забезпечують найкращу продуктивність. Крос-платформні - Flutter, React Native, Xamarin - єдина кодова база для кількох платформ. Гібридні - веб-застосунки всередині WebView (Ionic, Cordova). PWA (Progressive Web Apps) - веб-застосунки з нативними можливостями (офлайн, push-сповіщення).

8. Класифікація середовищ розробки мобільних застосунків

За платформою: Android Studio (офіційне IDE для Android), Xcode (iOS), Visual Studio (Xamarin). За підходом: нативні IDE з вбудованим емулятором і профайлером; крос-платформні (VS Code + Flutter/React Native); хмарні (CodeSandbox, Expo Snack). Середовища відрізняються за підтримкою платформ, інструментами відлагодження, інтеграцією з системами контролю версій та зручністю налаштування.

9. Класифікація та характеристика мобільних платформ

Android (Google) - відкрита ОС на базі Linux, мови розробки Kotlin та Java, частка ринку ~72%, розповсюджується через Google Play. iOS (Apple) - замкнута екосистема, мова Swift/Objective-C, частка ~27%, магазин App Store з суворою модерацією. HarmonyOS (Huawei) - власна платформа для пристроїв Huawei з підтримкою застосунків Android. KaiOS - легка платформа для кнопочкових телефонів на базі веб-технологій. Платформи відрізняються архітектурою, відкритістю API, вимогами до публікації та фрагментацією пристроїв.

5. Висновок

У ході виконання лабораторної роботи було розроблено Android-застосунок на мові Kotlin, що реалізує інтерфейс з двома фрагментами в рамках однієї Activity. Перший фрагмент (FirstFragment) містить два поля для введення чисел, групу радіо-кнопок для вибору однієї з чотирьох арифметичних операцій (+, -, ×, ÷) та кнопку «ОК». Другий фрагмент (SecondFragment) відображає результат обчислення і надає кнопку «Cancel» для повернення та очищення форми введення.

Передача даних між фрагментами реалізована через `SharedViewModel` з використанням `MutableLiveData`. `ViewModel` містить два поля: `resultText` для передачі результату та `shouldClear` для сигналу очищення форми. Такий підхід забезпечує повне розділення логіки між фрагментами без прямих посилань між ними і дозволяє виконувати необмежену кількість обчислень без перезапуску застосунку.

Реалізовано валідацію введених даних: Toast-повідомлення при неповному введенні та окреме повідомлення при спробі ділення на нуль. У результаті виконання роботи набуто практичні навички роботи з `Fragment`, `FragmentManager`, `FragmentTransaction`, а також з архітектурними компонентами `ViewModel` і `LivData`.